Benchmarking GNN Inference on the Intel Core Ultra NPU: A Latency, Quantization, and Energy Analysis

Yusuf Talha ARABACI
Dept. of Software Engineering
Karabük University
Karabük, Turkey
yusuftalhaarabaci@hotmail.com

Emrullah DEMİRAL
Dept. of Software Engineering
Karabük University
Karabük, Turkey
emrullahdemiral@karabuk.edu.tr

Ömer Faruk ACAR
Dept. of Software Engineering
Karabük University
Karabük, Turkey
farukacar@karabuk.edu.tr

***Abstract*—Neural Processing Units (NPUs) in client System-on-Chips (SoCs) target low-power inference, but their performance on sparse Graph Neural Networks (GNNs) remains underexplored. We benchmark 14 GNN, CNN, and transformer models across the CPU, integrated GPU (iGPU), and NPU of an Intel Core Ultra platform using OpenVINO and three Open Graph Benchmark datasets. The NPU accelerates dense vision models (MobileNetV2: 1.90 ms, 4.5–11.4× speedup over CPU) but yields negligible gains for memory-bound GNNs. NPU INT8 quantization causes compilation failures in three architectures and a 2.2× latency regression for SGC. CPU and iGPU backends operate within a 9–12.5 W power range, with INT8 providing at most an 18% energy reduction. The observed performance gap is consistent with a mismatch between the NPU's streaming-dataflow design and the irregular memory accesses of GNNs. We recommend the iGPU for GNNs and the NPU for FP32 vision models. The suite is available at https://yusufarbc.github.io/intel-npu-gnn-benchmarking/.**

***Index Terms*—NPU, GNN, Intel Core Ultra, Meteor Lake, OpenVINO, quantization, benchmarking, edge inference.**

I. Introduction

Neural Processing Units (NPUs) in client System-on-Chips (SoCs) are specialized hardware accelerators engineered for low-power, energy-efficient deep learning inference. In the Intel Meteor Lake processor architecture (e.g., Core Ultra 5 125H), the AI Boost NPU operates as a dedicated low-power engine alongside the multi-core CPU and integrated GPU (iGPU) within a nominal 28 W mobile client form factor. While such consumer-tier client SoCs are optimized for computer vision workloads, their practical performance on sparse, irregular Graph Neural Networks (GNNs) remains underexplored relative to high-power datacenter accelerators.

Unlike dense tensor operations, GNN execution combines sparse matrix multiplication (SpMM)—specifically sparse-times-dense matrix multiplication (SpDMM)—with dynamic feature gathering over irregular adjacency structures, yielding low arithmetic intensity and high sensitivity to data movement [1], [2]. As shown in Fig. 3, the exported GNN graphs contain 2–4× more shape-manipulation operators (Gather, Scatter, Reshape) than the vision graphs. These operators involve irregular index dependencies that conflict with the NPU's streaming-dataflow architecture. While prior works evaluate GNNs on server GPUs and custom ASICs [3]–[5], consumer client NPUs present a different design point: a restricted power envelope, shared system memory, compiler-managed data movement, and incomplete support for sparse or dynamic primitives [6], [7].

We develop a reproducible benchmarking suite evaluating 14 models across three Open Graph Benchmark (OGB) datasets and three Meteor Lake backends (CPU, iGPU, NPU) under OpenVINO 2024.1. The contribution is a boundary characterization of an accessible client NPU: it identifies where nominal accelerator availability translates into native execution, where quantization regresses, and where toolchain fallback can invalidate device-level comparisons. Our contributions include:

- A reproducible profiling protocol (100 iterations × 3 runs × 3 datasets) extracting latency, throughput, precision gain, CPU-fallback ratios, and ONNX Runtime execution traces.
- Cross-architectural latency comparisons demonstrating that the iGPU is the most effective accelerator for GNNs, whereas the NPU only excels on dense FP32 vision models.
- Empirical evidence showing INT8 quantization provides minimal or negative returns on the NPU for GNNs, with attention-based architectures failing compilation.
- Analysis of operator-level CPU fallbacks and density-scaling behavior under static-shape NPU compilation on a client platform.

II. Background

A. GNN Computation and Acceleration Landscape

Graph Neural Network architectures compute node representations via iterative neighborhood aggregation. Across spectral [8], [9], spatial [10], [11], attentional [12]–[14], and propagation-based [15], [16] models, a central computational primitive is sparse matrix–dense matrix multiplication (SpMM), combined with nonlinear feature transformations.

These operations exhibit irregular memory access patterns and poor spatial locality, stressing memory bandwidth and on-chip caches [1], [2], [17].

To alleviate these memory bottlenecks, domain-specific GNN accelerators such as HyGCN [3], EnGN [1], GRIP [4], and TT-GNN [5] implement dedicated hardware units for irregular edge gathering, custom scratchpad hierarchies, and crossbar interconnects. However, these custom ASICs require specialized non-commodity silicon. In contrast, commodity edge accelerators and client NPUs [18], [19] are designed around dense matrix multiplication engines optimized for regular systolic dataflows.

B. Intel Meteor Lake NPU Architecture

Intel Meteor Lake client SoCs integrate the AI Boost NPU on the SoC tile and share system memory with the CPU and iGPU across the SoC fabric [6], [7]. This client architecture makes end-to-end performance sensitive to data movement. Its streaming execution model favors continuous, predictable memory layouts (e.g., CNNs) but can incur overhead on irregular graph topologies with low data reuse. The OpenVINO NPU plugin compiles supported graph regions through operator fusion and layout transformations [20]. In the evaluated ONNX Runtime/OpenVINO path, operations not assigned to a supported accelerator subgraph remain on the host CPU; the profiling procedure therefore checks actual device assignment rather than inferring it from the requested provider.

C. Quantization and Edge Benchmarking

Low-precision integer quantization (INT8) is standard practice for accelerating edge inference [21]. However, quantizing GNNs is complicated by high dynamic activation ranges and non-uniform degree distributions across graph nodes [22]. Modern graph compilers apply operator fusion to minimize memory traffic [23], [24], but sparse indexing patterns frequently break fusion chains. MLPerf Inference covers inference systems, while GNNMark characterizes GNN training on GPUs [25], [26]; neither benchmark targets GNN inference on a Meteor Lake-class client NPU. Empirical evaluation with real-world graph datasets [27] therefore remains an open area of research.

III. Methodology

A. Experimental Platform

Experiments were conducted on an Intel Core Ultra 5 125H mobile processor (nominal 28 W processor base power, 3.60 GHz maximum turbo frequency, 16 GB LPDDR5x RAM, Windows 11 Build 26200). This is a power-constrained client platform rather than a server- or datacenter-class accelerator. The evaluated backends are: (1) the 14-core CPU (4 Performance, 8 Efficient, and 2 Low-Power Efficient cores), (2) the integrated Arc GPU (iGPU, Xe-LPG architecture, 7 Xe-cores), and (3) the AI Boost NPU (3720 series, VPUX37XX microarchitecture). OpenVINO's device identifier `GPU` refers to this integrated GPU; we use *iGPU* throughout the paper. The software stack uses OpenVINO 2024.1 and ONNX Runtime 1.18.

B. Benchmarked Models

Table I lists the 14 evaluated models. The GNN selection covers spectral (GCN, SGC), spatial (GraphSAGE, GIN), attentional (GAT, GATv2), propagation (APPNP), hybrid (GraphTransformer), and message-passing (MPNN) paradigms. These models (0.02M–0.20M parameters) represent edge-deployment scales where memory limits constrain size. Baselines include CNNs (ResNet50, MobileNetV2, EfficientNet-B0) and transformers (ViT-Tiny, BERT-Tiny). All models were exported to ONNX at FP32; INT8 variants were generated via ONNX Runtime dynamic quantization. GAT and GATv2 failed INT8 compilation due to unsupported dynamic scatter/gather operators and are evaluated in FP32 only.

TABLE I
Evaluated Models and Characteristics

Model	Category	Params	INT8
Graph Neural Networks			
GCN [8]	Spectral	0.10M	✓
GAT [12]	Attentional	0.11M	×
GATv2 [13]	Attentional	0.13M	×
GIN [11]	Expressive	0.09M	✓
GraphSAGE [10]	Spatial	0.18M	✓
SGC [9]	Simplified	0.02M	✓
APPNP [15]	Propagation	0.09M	✓
GraphTransformer [14]	Hybrid	0.18M	✓
MPNN [16]	Message-Passing	0.20M	✓
Dense Baselines			
ResNet50 [28]	CNN	25.5M	✓
MobileNetV2 [29]	CNN	3.5M	✓
EfficientNet-B0 [30]	CNN	5.3M	×
ViT-Tiny [31]	Vision Trans.	5.7M	×
BERT-Tiny [32]	NLP Trans.	4.4M	✓

× = INT8 compilation failed (unsupported operators/per-channel incompatibility). ✓ = INT8 benchmarked.

C. Datasets

We use three real-world Open Graph Benchmark (OGB) datasets [27] (Table II). Vision and NLP models receive synthetic inputs matching native shapes (e.g., $1\times3\times224\times224$ for CNNs). Dataset preprocessing and tensor extraction run once in a separate Python subprocess, before model warm-up and timing, because the dataset and inference stacks require potentially conflicting native-library environments. The subprocess serializes the prepared tensors to disk and exits; therefore, process startup, dataset loading, and serialization are excluded from the reported inference latency. Without isolation, loading would occur in the benchmark process and could introduce library conflicts and additional initialization variability, but it would not change the definition of steady-state inference latency because loading remains outside the timed region.

D. Quantization Methodology

All INT8 variants were generated via ONNX Runtime dynamic quantization; FP32 and INT8 are the two precisions

TABLE II
OGB DATASET CHARACTERISTICS

Dataset	Nodes	Edges	Features	Edges/node
ogbn-arxiv	169K	1.16M	128	6.9
ogbn-products	2.45M	61.9M	100	25.3
ogbn-proteins	87.6K	39.6M	8	451.7

evaluated in this study, not a claim about the complete set of precisions supported by the hardware. Dynamic quantization was selected over post-training static quantization (PTQ) because it requires no calibration dataset and avoids fixing activation ranges from one graph sample. Its runtime computation of activation quantization parameters can, however, introduce overhead and produce less readily fused graphs than PTQ. Section IV-B reports the resulting latency trade-offs; model accuracy after quantization is outside the scope of this systems characterization.

E. Measurement Protocol

Each configuration runs a 5-iteration warm-up and 100 timed iterations, repeated across three independent runs. We report the mean latency and per-iteration standard deviation. Bootstrap 95% confidence intervals (10,000 resamples) are calculated. All runs use ONNX Runtime graph optimizations (`ORT_ENABLE_EXTENDED`) under respective OpenVINO or CPU providers.

F. Artifacts and Metrics

We collect end-to-end inference latency (T_{inf} in ms), throughput ($1000/T_{\text{inf}}$ inferences/s), INT8/FP32 speedup ($T_{\text{FP32}}/T_{\text{INT8}}$), CPU-fallback ratios, and graph optimization speedup ($T_{\text{baseline}}/T_{\text{optimized}}$). Package power is measured via Intel SoCWatch (`-f sys -t 30 -r sum`) for GCN and MPNN on CPU and iGPU backends; NPU-specific power rail isolation was unsupported by the client platform telemetry. Arithmetic intensity (FLOP/byte) is computed via ONNX shape inference.

IV. RESULTS

A. Cross-Architectural Latency Comparison

Inference latency is the mean wall-clock time in milliseconds (T_{inf}) for one batch-size-one forward pass after warm-up. It includes execution-provider dispatch and required host–device transfers, but excludes dataset loading, preprocessing, model conversion, and one-time graph compilation. For sequential batch-size-one execution, the corresponding throughput is $1000/T_{\text{inf}}$ inferences/s; latency is reported because it is the direct responsiveness metric for the targeted interactive edge setting. Fig. 1 compares FP32 latencies across backends. The iGPU achieves the lowest latency for most GNNs, e.g., GraphTransformer (6.03 ms iGPU vs. 10.72 ms NPU vs. 10.69 ms CPU) and APPNP (7.88 ms iGPU vs. 11.75 ms NPU vs. 11.82 ms CPU). GCN reaches 6.94 ms on iGPU and 7.91 ms on NPU. The NPU shows near-parity (within 6%) to the CPU for GNNs but is consistently outperformed by the iGPU. Conversely, the NPU accelerates dense vision workloads: MobileNetV2 achieves 1.90 ms (4.5× speedup over CPU), ResNet50 runs at 3.94 ms (8.0×), and ViT-Tiny reaches 9.10 ms (11.4× over CPU). This divergence is analyzed in Section V.

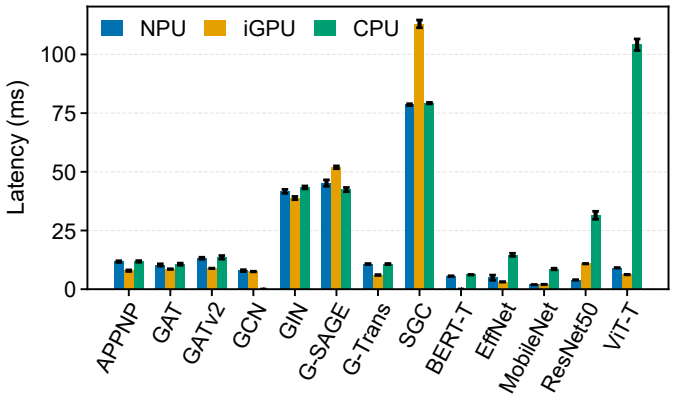

Fig. 1. FP32 latency comparison across CPU, iGPU, and NPU backends. Error bars indicate per-iteration standard deviation. GNNs show near-parity across backends; dense vision models exhibit significant NPU advantage.

B. INT8 Quantization: Performance Impact

Table III shows INT8 speedups on the NPU. GraphTransformer benefits (1.21×), consistent with its larger share of dense attention kernels. Other GNNs show marginal speedups (GCN: 1.04×, GIN: 1.03×, GraphSAGE: 1.05×), indicating that quantization alone does not remove the observed memory bottlenecks. APPNP (0.84×) and SGC show regressions; SGC's INT8 latency (173.90 ms) is 2.2× slower than FP32 (78.59 ms, 0.45× speedup), an outcome associated with additional dispatch overhead during K-step propagation. Device-assignment outcomes vary by model. MobileNetV2 INT8 on the requested NPU runs at 5.38 ms versus 1.90 ms in FP32 and matches its 5.46 ms CPU INT8 latency, indicating CPU execution. Profiling confirms a small host component for BERT-Tiny INT8 (154–231 μs CPU execution). ResNet50 remains distinct from CPU timing (4.98 ms requested NPU vs. 18.50 ms CPU), while EfficientNet-B0 fails INT8 compilation.

TABLE III
INT8/FP32 SPEEDUP ON NPU (MEAN ACROSS OGB DATASETS)

Model	FP32 (ms)	INT8 (ms)	Speedup
GraphTransformer	10.72	8.90	**1.21**×
GCN	7.91	7.58	**1.04**×
GraphSAGE	45.21	42.95	1.05×
GIN	41.68	40.55	1.03×
APPNP	11.75	13.96	0.84×
SGC	78.59	173.90	0.45×
GAT	10.32	—	—†
GATv2	13.19	—	—†

Speedup $>$ 1.0 indicates INT8 faster. †INT8 compilation failed due to unsupported dynamic scatter/gather operations. EfficientNet-B0 also failed INT8 compilation. Dense vision models are omitted from this table due to silent CPU fallback under INT8 (see Section IV-B).

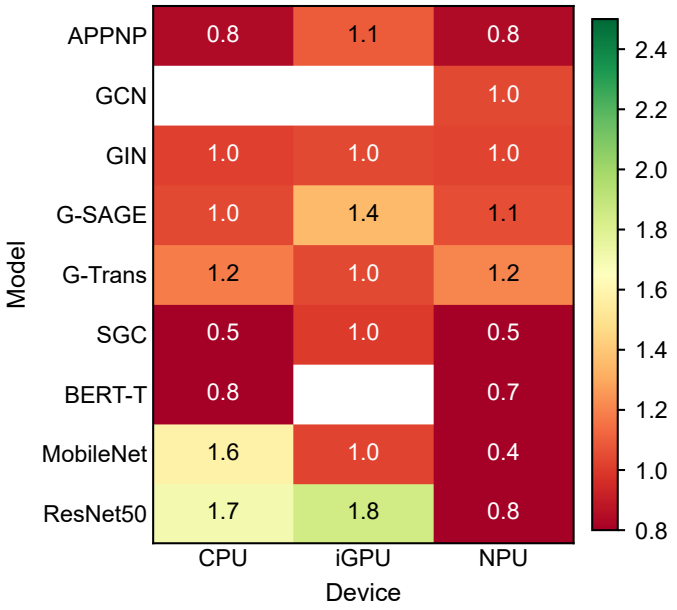

Fig. 2. INT8 speedup heatmap across models and devices. Values below 1.0 (red) indicate INT8 regression. The iGPU consistently benefits; NPU and CPU show mixed results.

C. Operator Composition Analysis

Fig. 3 reports the fraction of nodes in each exported ONNX graph; it is a structural composition, not an execution-time breakdown. Consequently, a large *Other* segment does not imply that most latency is spent there. *Other* groups inexpensive nonlinear activations (e.g., ReLU, GELU, Sigmoid), normalization, elementwise arithmetic (Add, Sub, Mul), and constant/slicing nodes that would otherwise make the legend unreadable. The relevant contrast is that GNN graphs contain 2–4× more Gather, Scatter, Reshape, and Concat nodes than vision graphs, whereas vision graphs contain a larger fraction of regular Conv nodes. ViT-Tiny (5.7M parameters) achieves an 11.4× speedup, whereas GraphTransformer (0.18M parameters) shows no NPU speedup (10.72 ms NPU vs. 10.69 ms CPU). This contrast is consistent with their different operator structures, but graph composition alone does not isolate the cause of the latency difference.

D. CPU Fallback and Operator Coverage

Table IV replaces the visually uniform fallback heatmap and reports only the exceptions that affect interpretation. Most successfully compiled FP32 graphs showed no measured CPU fallback. MPNN is the material exception: its dynamic index accumulation operations (`index_add_` and `scatter_mean`) were not assigned to the NPU, so the requested NPU configuration executed on the CPU. BERT-Tiny INT8 showed a small host component associated with embedding lookup, while GAT, GATv2, and EfficientNet-B0 produced no INT8 executable and therefore have no fallback percentage. This distinction prevents a compilation failure from being presented as a measured 100% fallback.

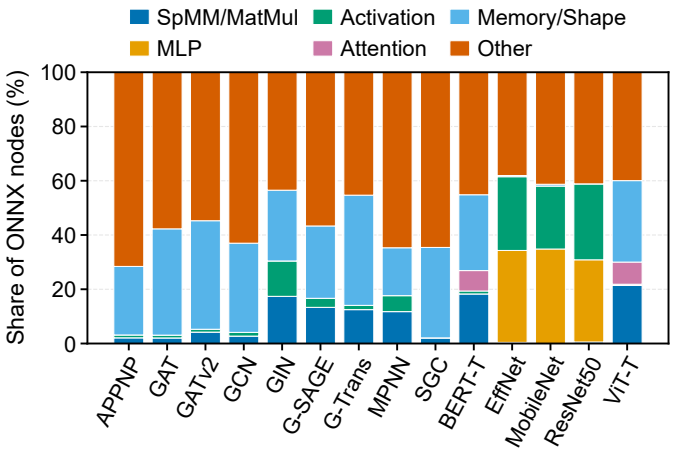

Fig. 3. ONNX graph-node composition (not execution time). *Other* groups activations, normalization, elementwise arithmetic, and constant/slicing nodes; the key contrast is the larger memory/shape fraction in GNNs.

TABLE IV
OBSERVED DEVICE-ASSIGNMENT EXCEPTIONS

Model	Configuration	Observed outcome
MPNN	NPU FP32	CPU execution
BERT-Tiny	NPU INT8	Partial CPU execution
GAT, GATv2	NPU INT8	Compilation failed
EfficientNet-B0	NPU INT8	Compilation failed

E. Optimization Speedup and Roofline Analysis

Fig. 4 shows optimization speedups (extended vs. disabled ORT). NPU optimization gains are small (1.0–1.05×), whereas iGPU optimization achieves 1.1–1.25× speedup via OpenVINO kernel fusion. Fig. 5 presents a throughput-intensity roofline. GNNs cluster in the low-intensity (0.1–10 FLOP/byte) and low-throughput (1–3 GFLOP/s) region, well below the 120 GB/s theoretical LPDDR5x reference ceiling derived from the platform memory configuration [6]. In contrast, ResNet50 and MobileNetV2 utilize the NPU's compute pipelines more effectively (8–20 GFLOP/s at 18–72 FLOP/byte). The reference line is not an NPU-specific measured bandwidth limit; all three devices share the memory subsystem, and achievable bandwidth depends on fabric and workload behavior.

F. Graph Density, Size Scaling, and Static-Shape Compilation

Figs. 6 and 7 separate two effects. Across the three OGB datasets (6.9–451.7 edges/node), NPU latency has no detectable correlation with graph density ($r = -0.00$, $p = 0.993$). In the APPNP FP32 size sweep at a fixed density of 7 edges/node, however, NPU latency increases from 5.92 ms at 512 nodes to 36.03 ms at 8192 nodes (6.1× latency for 16× as many nodes). Each point is a separately compiled fixed-shape graph, so this experiment characterizes size scaling across compiled shapes rather than dynamic-shape execution within one compiled model.

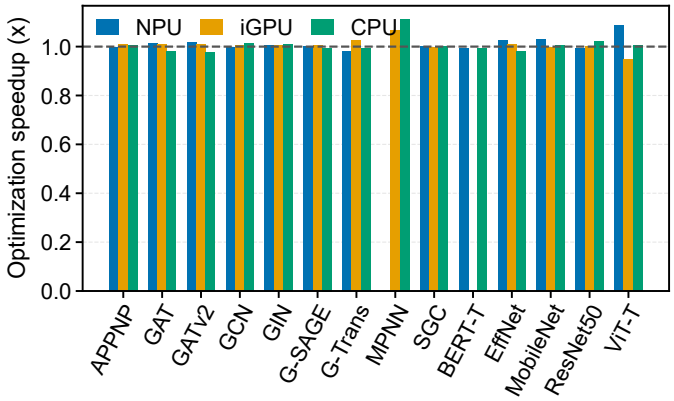

Fig. 4. ORT optimization speedup by device, averaged across datasets and available precisions for each model. The iGPU shows larger gains from graph-level optimization for GNNs; NPU optimization returns are uniformly modest.

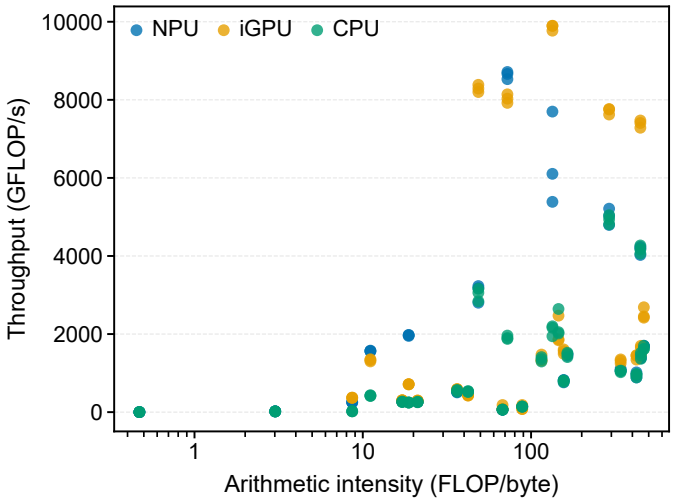

Fig. 5. Computational efficiency (throughput vs. arithmetic intensity). GNNs occupy the memory-bound region (low intensity, low throughput); vision models on NPU achieve higher operational intensity.

G. Energy Consumption Analysis

Table V shows package-level power collected via SoCWatch (`-f sys -t 30 -r sum`). The iGPU draws slightly higher power than the CPU for GCN (12,482 mW vs. 11,629 mW, +7.3%) but reduces latency (6.94 ms vs. 7.73 ms), resulting in comparable energy per inference (86.6 vs. 89.9 mJ). For MPNN, which falls back to the CPU, package power (9,357 vs. 9,473 mW) and latencies (20.23 vs. 22.03 ms) remain nearly identical. CPU INT8 quantization for GCN reduces power (11,629 to 9,675 mW, –16.8%) with stable latency (7.73 to 7.59 ms, –1.8%), decreasing energy by 18.4%. Conversely, MPNN CPU INT8 increases latency to 32.94 ms (+63% over FP32: 20.23 ms) and energy to 301.1 mJ (+59% over FP32: 189.3 mJ), despite a slight power decrease (9,139 vs. 9,357 mW). For GNNs, INT8 energy benefits are model-dependent and not guaranteed. NPU power rail isolation was unavailable under the current PMT interface.

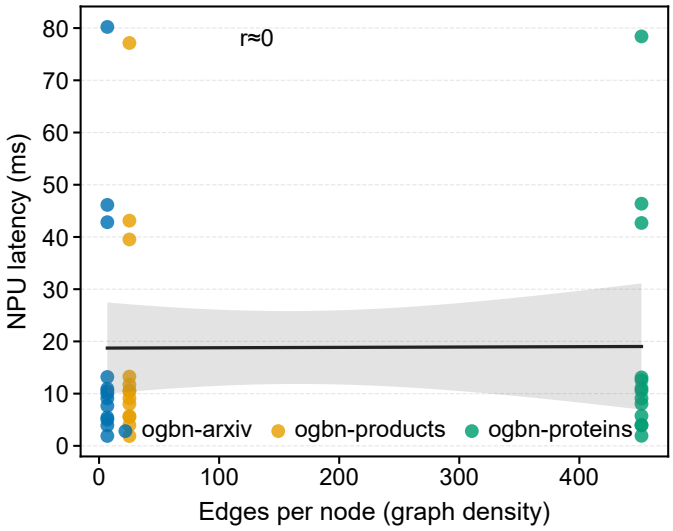

Fig. 6. NPU latency vs. graph density. The flat regression line ($r \approx 0$) shows no detectable relationship in the evaluated sweep and is consistent with static-shape compilation.

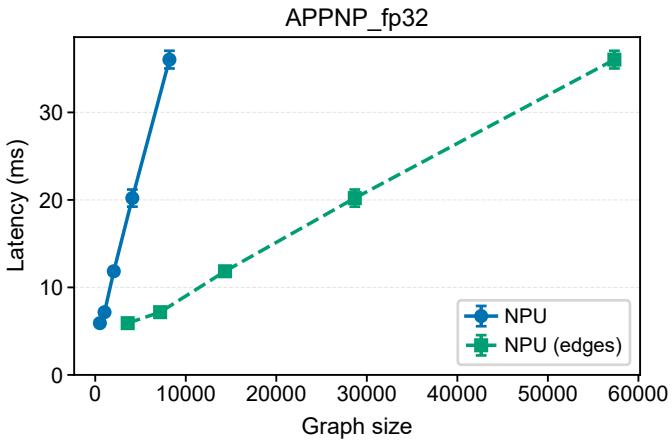

Fig. 7. APPNP FP32 latency scaling on NPU across graph sizes (nodes N and edges E at 7 edges/node). Solid line shows scaling by nodes; dashed line shows scaling by edges. Error bars show run-to-run standard deviation; each point represents a separately compiled fixed-shape graph.

TABLE V
PACKAGE POWER AND ENERGY PER INFERENCE (SOCWATCH, 30 S COLLECTION WINDOW)

Model	Config	Avg Power (mW)	Latency (ms)	Energy/Inf. (mJ)
GCN	CPU FP32	11,629	7.73	89.9
GCN	CPU INT8	9,675	7.59	73.4
GCN	iGPU FP32	12,482	6.94	86.6
MPNN	CPU FP32	9,357	20.23	189.3
MPNN	CPU INT8	9,139	32.94	301.1
MPNN	iGPU FP32	9,473	22.03	208.7
MPNN	iGPU INT8	9,161	32.48	297.6

Latency is the mean time over 100 iterations × 3 repeats across three OGB datasets. Energy/inf. is estimated as Avg Power × Mean Latency over the 30 s SoCWatch window, representing an upper bound that includes background system activity. NPU power isolation was unsupported by the PMT interface.

V. Discussion

A. Why INT8 Underperforms for GNNs on NPU

The limited INT8 gains are consistent with the combination of low arithmetic intensity, irregular indexing, dynamic-quantization overhead, and compiler coverage. Precision reduction lowers arithmetic cost but does not remove sparse Gather/Scatter traffic; the exported GNNs also contain 2–4× more shape-manipulation nodes than the vision models. Indirect indexing can restrict fusion opportunities [23], consistent with the smaller optimization gains measured on the NPU.

Outcomes are model-specific: MobileNetV2 INT8 timing indicates CPU execution, BERT-Tiny INT8 includes a small host component, MPNN requested on NPU at FP32 executes on CPU, and three INT8 graphs fail compilation. These exceptions make execution traces essential; the experiment associates them with operator structure and toolchain behavior but does not isolate one hardware or compiler cause.

B. Platform Maturity Assessment

Under OpenVINO 2024.1, the Meteor Lake NPU primarily benefits the evaluated dense vision models (4.5–11.4× over CPU), while the iGPU leads the GNN measurements. The iGPU result is consistent with its cache and shared-memory access plus larger measured OpenVINO optimization gains (1.1–1.25× vs. 1.0–1.05× on NPU). Memory-system behavior and compiler mapping are plausible contributors, but this benchmark does not isolate their effects.

C. Comparison with Prior Work

Custom GNN accelerators like HyGCN [3] ($10^3\times$ over CPU), EnGN [1] ($1.8 \times 10^3\times$ over CPU), and GRIP [4] (17× over CPU) rely on hardware-native sparse engines and dedicated memory hierarchies. By contrast, the Meteor Lake NPU provides 0.9–1.2× acceleration for GNNs over the CPU. This performance delta reflects conflicting optimization targets: GNN ASICs optimize for sparse dataflows, whereas consumer NPUs target dense tensor operations. This trend matches observations in the wider edge AI landscape [19].

D. Limitations

Single-Platform Scope: All measurements were collected on a single Intel Core Ultra 5 125H processor (4P+8E+2LPE cores, 7 Xe-cores iGPU, one NPU tile). Results may vary across Meteor Lake configurations with different core/memory setups (LPDDR5x vs. DDR5) and subsequent architectures (Lunar Lake, Arrow Lake) due to differences in memory bandwidth.

Statistical Reporting: Latencies represent arithmetic means over 100 iterations × 3 independent runs. Although 95% bootstrap confidence intervals show narrow typical widths of 0.1–0.5 ms, formal paired hypothesis tests were not conducted. Small latency variations, such as the ∼6% CPU–NPU gap on GNNs, are not assessed for statistical significance, and rankings may be affected by run-to-run laptop environment variability (DVFS, thermal throttling).

Power Measurement Constraints: SoCWatch measurements (Table V) cover only two models (GCN, MPNN) and two backends (CPU, iGPU). NPU power measurement was precluded by PMT hardware counter definitions, which expose NPU temperature and bandwidth (VPU_MEMORY_BW) but lack direct power rail counters, causing NPU-targeted sessions to produce only raw ETW files without power data.

Energy-Per-Inference Estimation: Energy per inference (Avg Power × Mean Latency) assumes stationary power across the 30 s collection window. This linear approximation does not isolate active inference energy from background system activity (display refresh, OS scheduling), serving only as an upper-bound estimate.

Compiler and Input Constraints: The benchmark uses fixed-shape inputs consistent with static-graph NPU compilation. Quantization compilation failures in GAT, GATv2, and EfficientNet-B0 could not be mitigated because the ONNX Runtime dynamic quantization API lacks the fine-grained control needed for manual per-operator quantization.

Workload Representativeness: The selected GNN architectures include established models that remain common reference points for systems research, and the OGB datasets provide standardized, real-world graphs with widely varying size and density. Nevertheless, the suite does not cover newer large graph transformers, heterogeneous graphs, temporal graphs, or recommendation workloads. The results should therefore be interpreted as a characterization of representative foundational inference kernels on this client NPU, not as a comprehensive ranking of current GNN applications.

Software Stack Versioning: Evaluation was limited to OpenVINO 2024.1 and ONNX Runtime 1.18. Since the OpenVINO NPU plugin is under active development, subsequent driver and software updates may expand operator coverage and improve GNN performance metrics.

VI. Conclusion

Across 14 models and three OGB datasets, the Meteor Lake NPU accelerates dense FP32 vision workloads by 4.5–11.4× over CPU but provides little latency benefit for the evaluated GNNs. INT8 outcomes include CPU execution, compilation failure, and a 2.2× SGC regression; NPU power could not be isolated. These results support using the NPU for compatible dense FP32 models and the iGPU for the tested GNNs, with execution traces checked for every quantized configuration. The suite is available at https://yusufarbc.github.io/intel-npu-gnn-benchmarking/.

Acknowledgment

Google Gemini 3.6 Flash was used for initial code scaffolding of the evaluation scripts described in Section III and for language-editing suggestions in Sections I, II, IV, and V. The authors reviewed and revised all AI-assisted material; the system did not generate experimental data, numerical results, figures, interpretations, or conclusions.

REFERENCES

[1] S. Liang, Y. Wang, C. Liu, L. He, H. LI, D. Xu, and X. Li, "Engn: A high-throughput and energy-efficient accelerator for large graph neural networks," *IEEE Transactions on Computers*, vol. 70, no. 9, pp. 1511–1525, Sep. 2021.

[2] A. Auten, M. Tomei, and R. Kumar, "Hardware acceleration of graph neural networks," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, July 2020, pp. 1–6.

[3] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, "Hygcn: A gcn accelerator with hybrid architecture," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb 2020, pp. 15–29.

[4] K. Kiningham, P. Levis, and C. Ré, "Grip: A graph neural network accelerator architecture," *IEEE Transactions on Computers*, vol. 72, no. 4, pp. 914–925, April 2023.

[5] Z. Qu, D. Niu, S. Li, H. Zheng, and Y. Xie, "Tt-gnn: Efficient on-chip graph neural network training via embedding reformation and hardware optimization," in *2023 56th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct 2023, pp. 452–464.

[6] W. Gomes, "Meteor lake and arrow lake: Intel next gen 3d client architecture platform with foveros," Hot Chips 34, 2022. [Online]. Available: https://hc34.hotchips.org/program/conference/

[7] Intel Corporation, "Heterogeneous ai powerhouse: Unveiling the hardware and software foundation of intel core ultra processor for the edge," Intel Corporation, White Paper Document 817736-1.0, 2024. [Online]. Available: https://www.intel.com/content/www/us/en/content-details/817736/

[8] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations*, 2017.

[9] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 6861–6871.

[10] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 1025–1035.

[11] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *International Conference on Learning Representations*, 2019.

[12] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations*, 2018.

[13] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *International Conference on Learning Representations*, 2022.

[14] Y. Shi, Z. Huang, S. Feng, H. Zhong, W. Wang, and Y. Sun, "Masked label prediction: Unified message passing model for semi-supervised classification," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, 2021, pp. 1548–1554.

[15] J. Gasteiger, A. Bojchevski, and S. Günnemann, "Predict then propagate: Graph neural networks meet personalized pagerank," in *International Conference on Learning Representations*, 2019.

[16] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 1263–1272.

[17] S. Abadal, A. Jain, R. Guirado, J. López-Alonso, and E. Alarcón, "Computing graph neural networks: A survey from algorithms to accelerators," *ACM Comput. Surv.*, vol. 54, no. 9, Oct. 2021.

[18] A. Reuther, P. Michaleas, M. Jones, V. Gadepally, S. Samsi, and J. Kepner, "Ai and ml accelerator survey and trends," in *2022 IEEE High Performance Extreme Computing Conference (HPEC)*, Sep. 2022, pp. 1–10.

[19] R. Singh and S. S. Gill, "Edge ai: A survey," *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023.

[20] OpenVINO Toolkit Core Team, *OpenVINO Toolkit 2024: NPU Device*, Intel Corporation, 2024. [Online]. Available: https://docs.openvino.ai/2024/openvino-workflow/running-inference/inference-devices-and-modes/npu-device.html

[21] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[22] S. A. Tailor, J. Fernandez-Marques, and N. D. Lane, "Degree-quant: Quantization-aware training for graph neural networks," in *International Conference on Learning Representations*, 2021.

[23] W. Niu, J. Guan, Y. Wang, G. Agrawal, and B. Ren, "Dnnfusion: accelerating deep neural networks execution with advanced operator fusion," in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, ser. PLDI 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 883–898.

[24] Z. Zhang, "Unified operator fusion for heterogeneous hardware in ml inference frameworks," in *2025 6th International Conference on Big Data & Artificial Intelligence & Software Engineering (ICBASE)*, July 2025, pp. 246–250.

[25] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, B. Bilangar *et al.*, "Mlperf inference benchmark," in *Proceedings of the 47th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 446–459.

[26] T. Baruah, K. Shivdikar, S. Dong, Y. Sun, S. A. Mojumder, K. Jung, J. L. Abellán, Y. Ukidave, A. Joshi, J. Kim, and D. Kaeli, "Gnnmark: A benchmark suite to characterize graph neural network training on gpus," in *2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, March 2021, pp. 13–23.

[27] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open graph benchmark: Datasets for machine learning on graphs," *Advances in Neural Information Processing Systems*, vol. 33, pp. 22 118–22 133, 2020.

[28] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[29] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520.

[30] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning*, 2019, pp. 6105–6114.

[31] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2021.

[32] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," *arXiv preprint arXiv:1908.08962*, 2019.